

Computer Vision

Image Segmentation



Image Segmentation

- Group pixels into meaningful or perceptually similar regions
- Task of grouping together similar (in semantic content) pixels in an image.

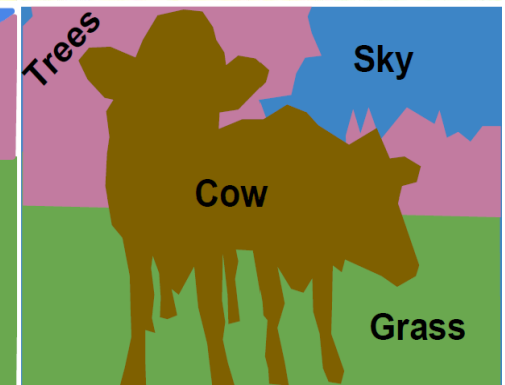
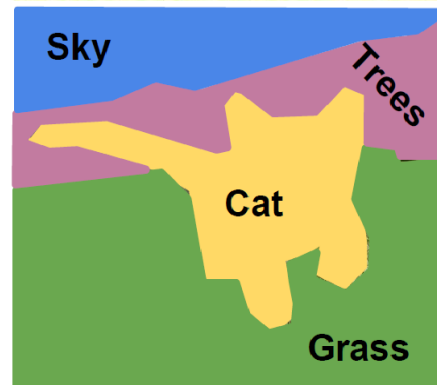
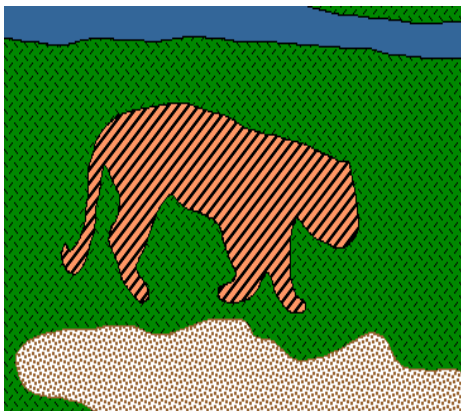


Image Segmentation

- How to formulate this using DNNs? Cast as a pixel-wise classification problem!
- For each training image, each pixel is labelled with a semantic category. Quite annotation-intensive!

Image Segmentation

- Segmentation methods: Watershed, Graph Cut, Normalized Cut, Mean Shift, K-means etc.
- Classical segmentation methods inspired early versions of DL based methods for object detection
- Various architectures in recent years: FCN, SegNet, U-Net, PSP-Net, DeepLab and Mask R-CNN

Segmentation using clustering

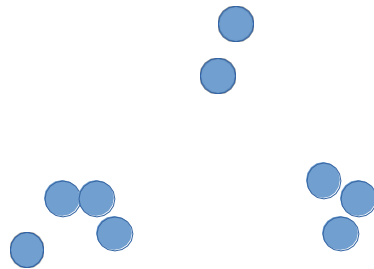
- Kmeans

K-means algorithm

$$\operatorname{argmin}_{S, \mu_i, i=1..K} \sum_{i=1}^K \sum_{x \in S_i} \|x - \mu_i\|^2$$

Partition the data into K sets $S = \{S_1, S_2, \dots, S_K\}$ with corresponding centers μ_i

Partition such that **variance** in each partition is **as low as possible**

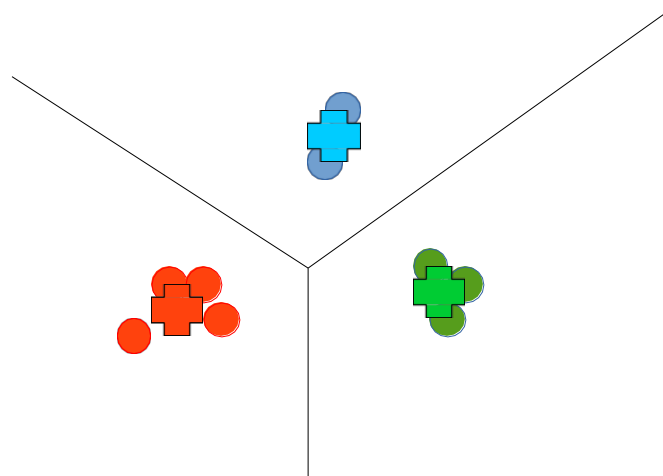


K-means algorithm

$$\operatorname{argmin}_{S, \mu_i, i=1..K} \sum_{i=1}^K \sum_{x \in S_i} \|x - \mu_i\|^2$$

Partition the data into K sets $S = \{S_1, S_2, \dots, S_K\}$ with corresponding centers μ_i

Partition such that **variance** in each partition is **as low as possible**



K-means algorithm

1. Initialize K centers μ_i (usually randomly)

K-means algorithm

1. Initialize K centers μ_i (usually randomly)

2. Assign each point x to its nearest center:

$$S^t = \operatorname{argmin}_S \sum_{i=1}^K \sum_{x \in S_i} \|x - \mu_i\|^2$$

K-means algorithm

1. Initialize K centers μ_i (usually randomly)

2. Assign each point x to its nearest center:

$$S^t = \operatorname{argmin}_S \sum_{i=1}^K \sum_{x \in S_i} \|x - \mu_i\|^2$$

3. Update cluster centers as the mean of its members

K-means algorithm

1. Initialize K centers μ_i (usually randomly)

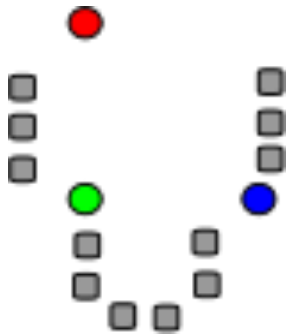
2. Assign each point x to its nearest center:

$$S^t = \underset{S}{\operatorname{argmin}} \sum_{i=1}^K \sum_{x \in S_i} \|x - \mu_i\|^2$$

3. Update cluster centers as the mean of its members

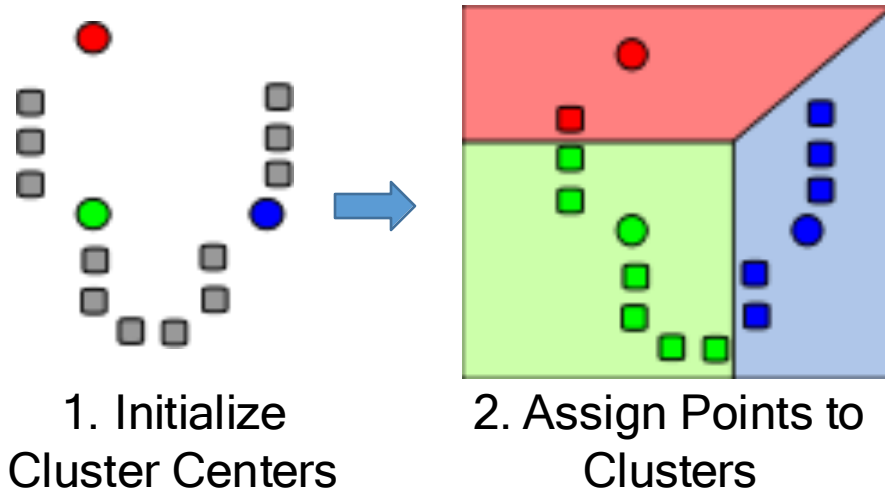
4. Repeat 2-3 until convergence ($t = t+1$)

K-means clustering

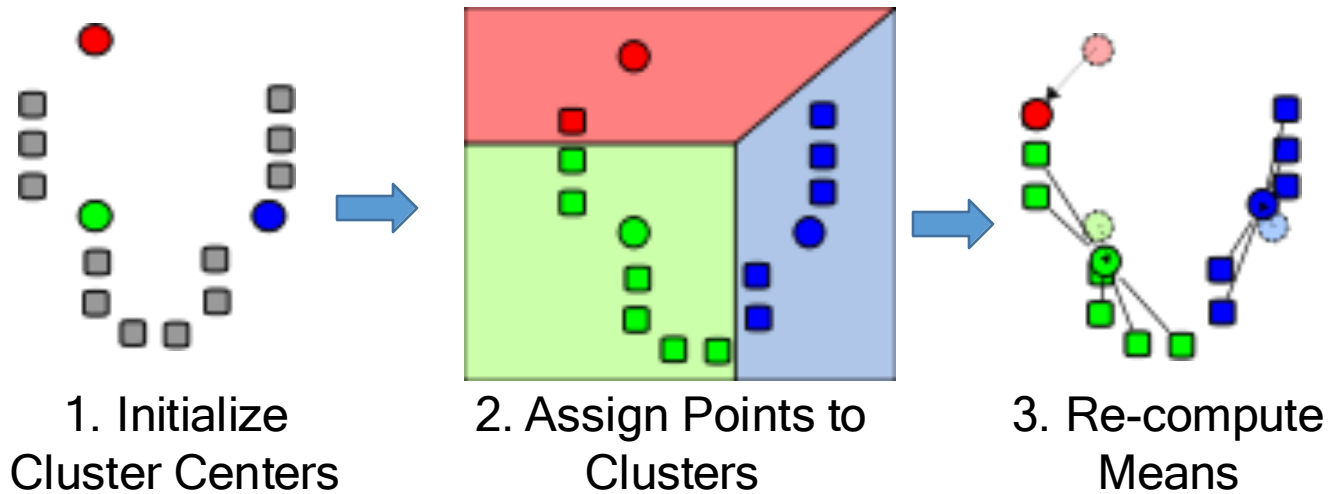


1. Initialize
Cluster Centers

K-means clustering



K-means clustering



K-means clustering

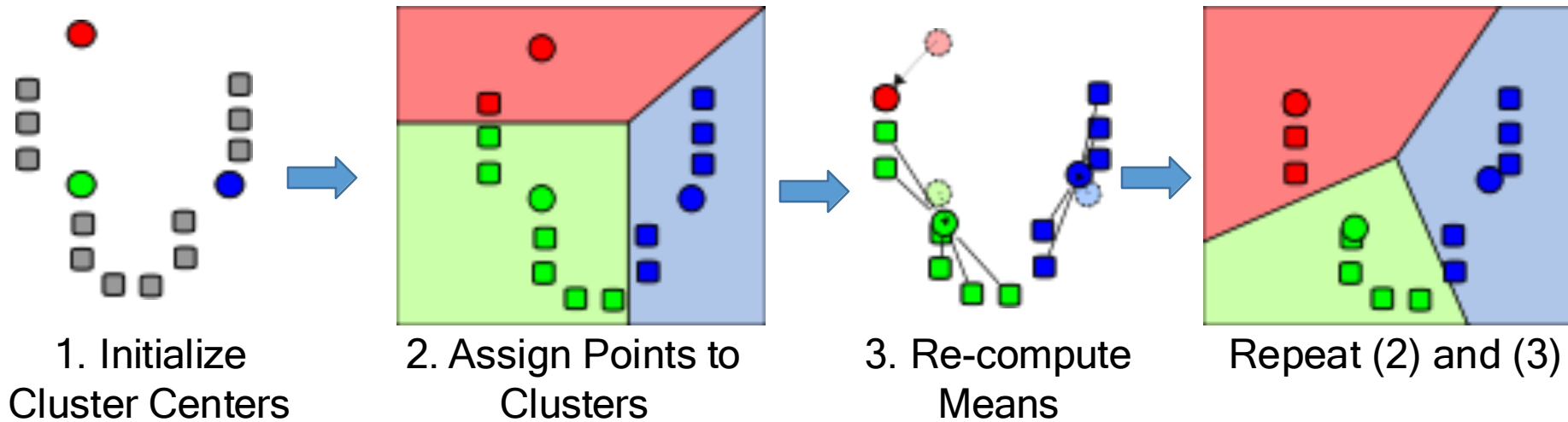


Image Segmentation by K-Means

- Select a value of K
- Select a feature vector for every pixel (color, texture, position, or combination of these etc.)
- Define a similarity measure between feature vectors (Usually Euclidean Distance).
- Apply K-Means Algorithm.
- Merge any components of size less than some threshold to an adjacent component that is most similar to it.

K-means clustering using intensity alone and color alone

Image



Clusters on intensity



Clusters on color



There is no requirement that clusters be spatially localized and they're not

K-Means pros and cons

- Pros

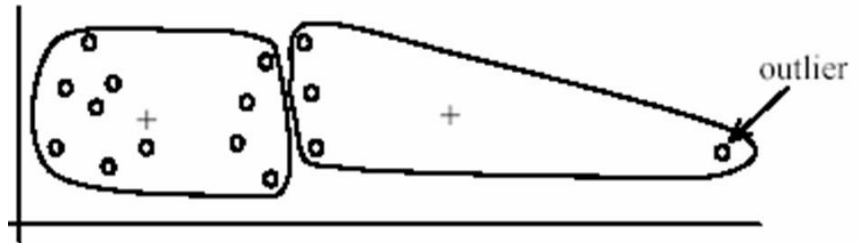
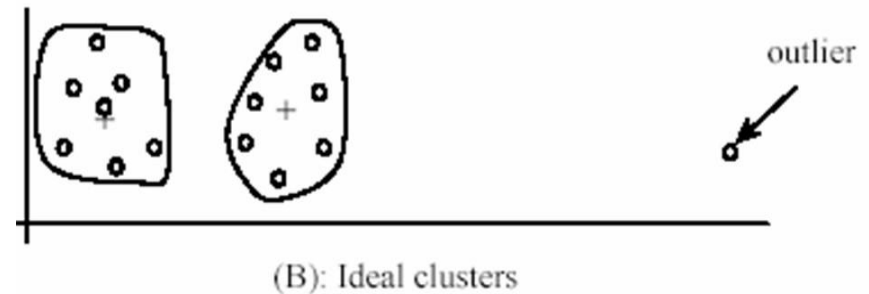
- Simple and fast
- Easy to implement

- Cons

- Need to choose K
- Sensitive to outliers

- Usage

- Unsupervised clustering
- Rarely used for pixel segmentation

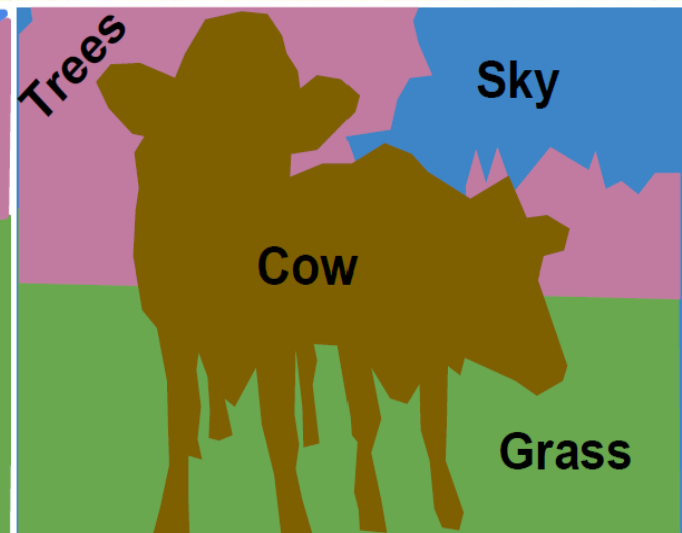
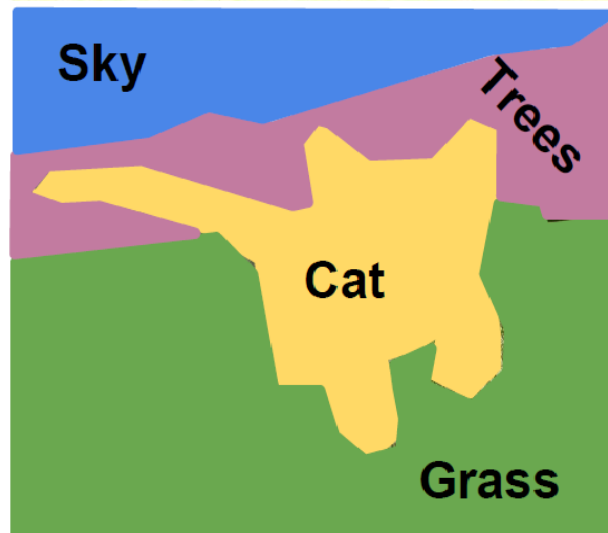


Semantic Segmentation

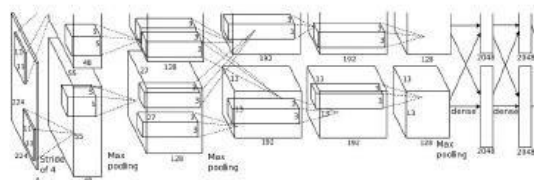
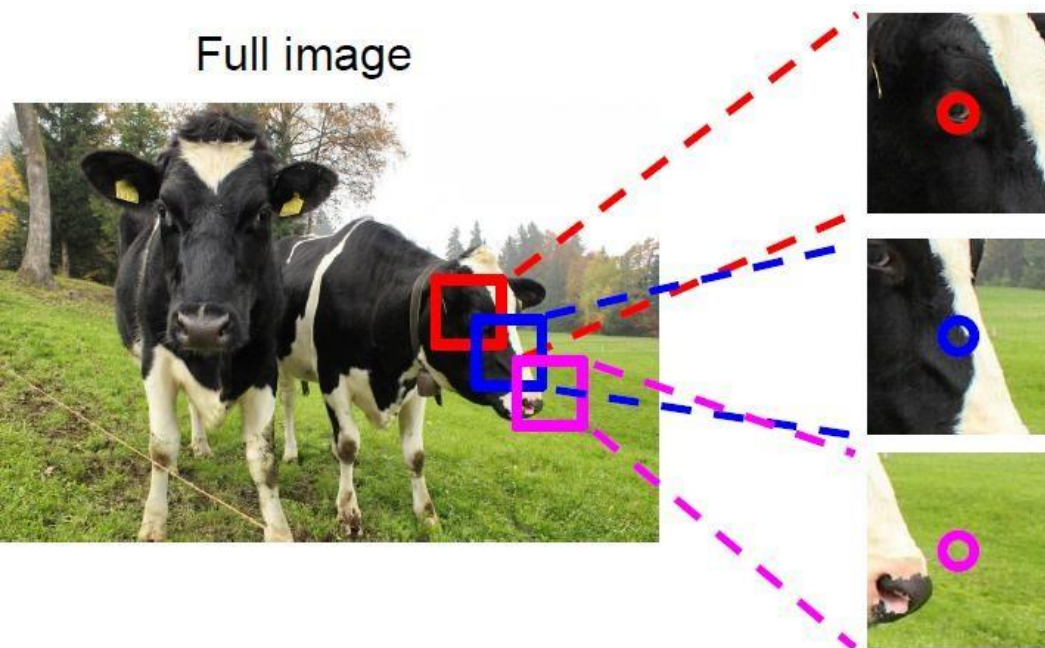
These images are CC0 public domain [source1](#) [source2](#)

Label each pixel in the image with a category label

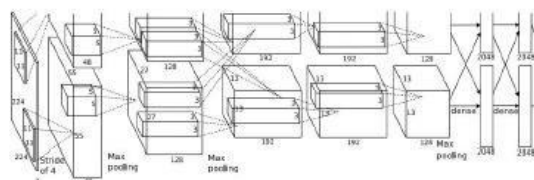
Don't differentiate instances, only care about pixels



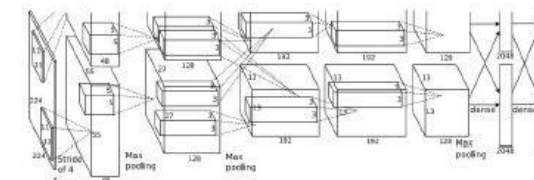
Semantic Segmentation: Sliding Window



Cow

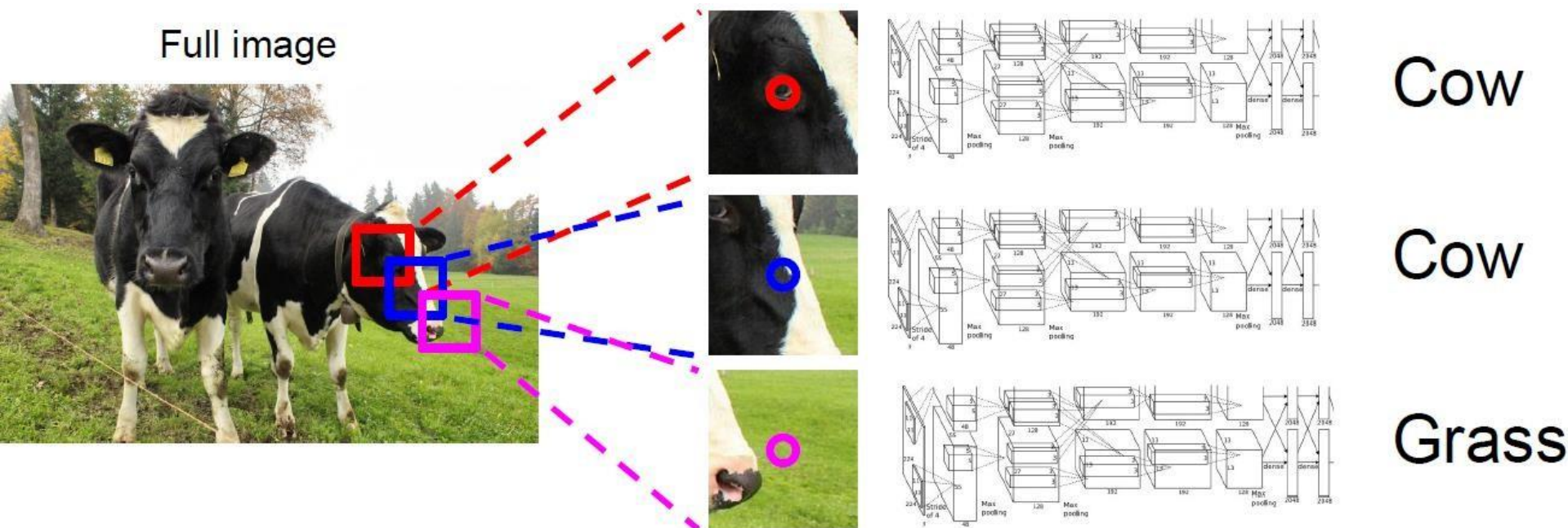


Cow



Grass

Semantic Segmentation: Sliding Window



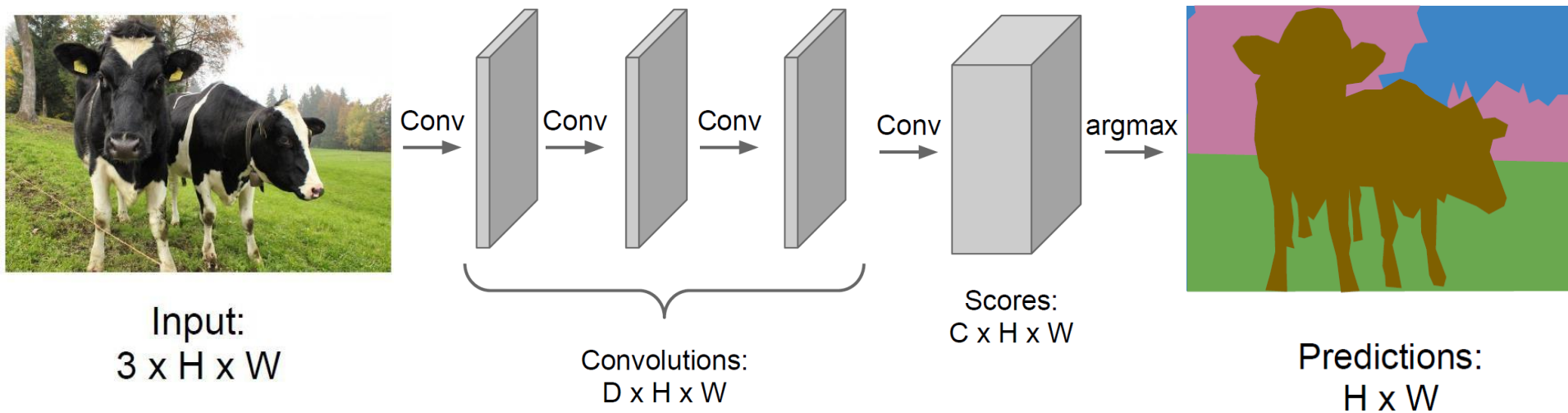
Problem:

Very inefficient!

Not reusing shared features between overlapping patches

Semantic Segmentation: Fully Convolutional Network

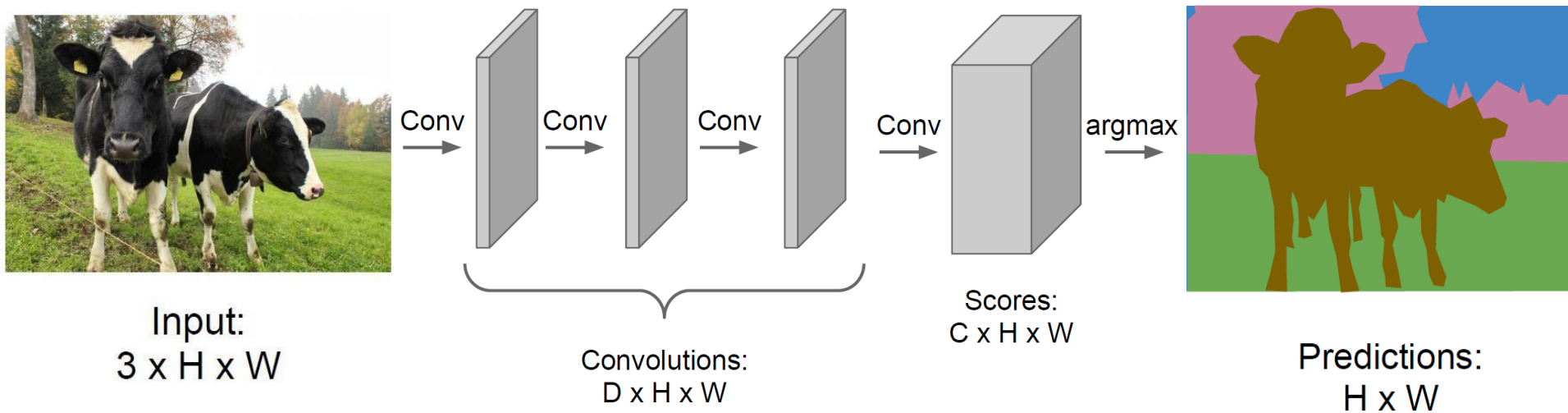
Design a network as a bunch of convolutional layers to make predictions for pixels all at once!



- Image classification architectures perform downsampling as they go deeper $\Rightarrow$ fully convolutional output will have lower resolution than input.

Semantic Segmentation: Fully Convolutional

Design a network as a bunch of convolutional layers
to make predictions for pixels all at once!



Problem:

Convolutions at original image resolution will be very expensive

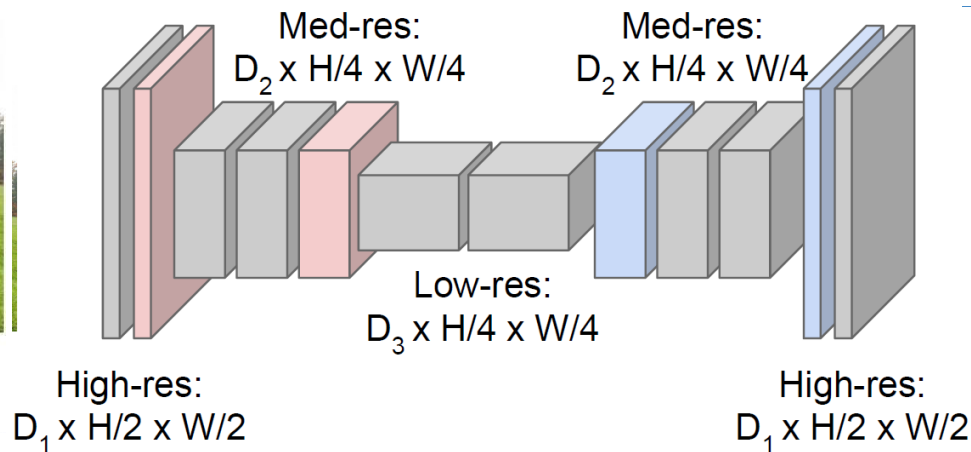
...

Semantic Segmentation: Fully Convolutional

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Input:
 $3 \times H \times W$



Predictions:
 $H \times W$

Semantic Segmentation: Fully Convolutional

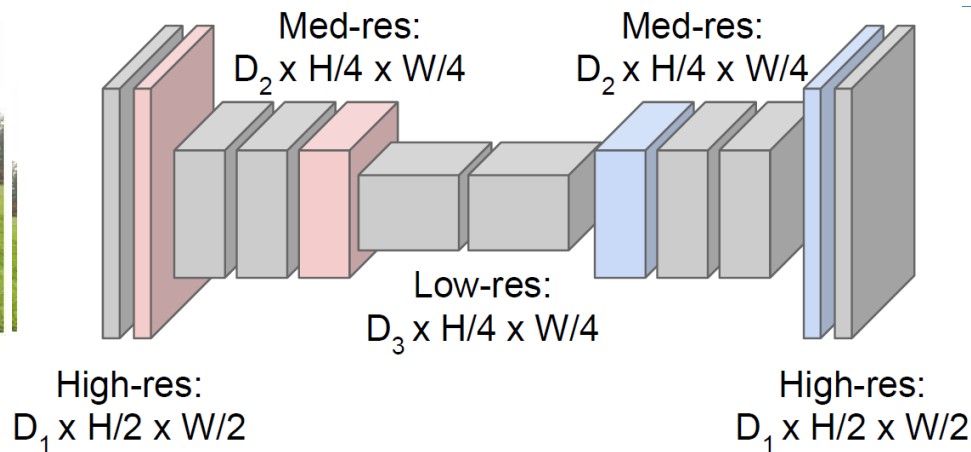
Downsampling:
Pooling, strided
convolution

Design network as a bunch of
convolutional layers, with **downsampling**
and **upsampling** inside the network!

Upsampling:
??



Input:
 $3 \times H \times W$



Predictions:
 $H \times W$

Semantic Segmentation: Fully Convolutional

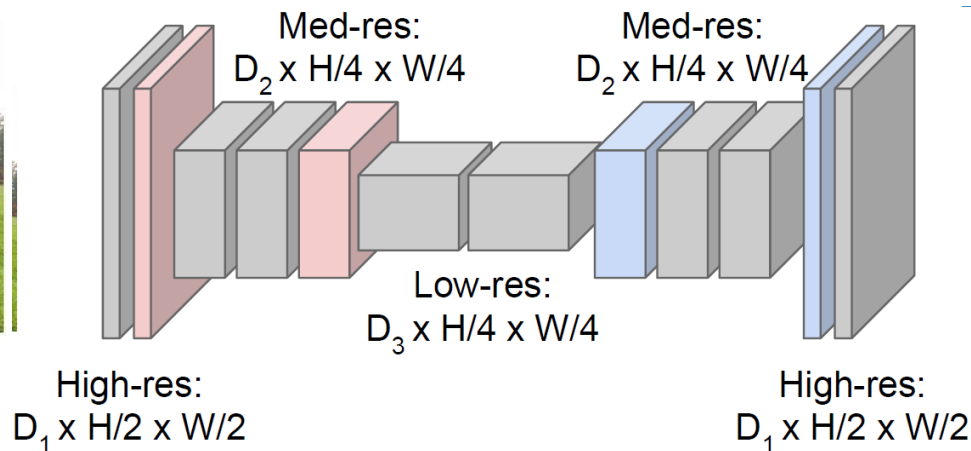
Downsampling:
Pooling, strided
convolution

Design network as a bunch of
convolutional layers, with **downsampling**
and **upsampling** inside the network!

Upsampling:
Unpooling or
strided transpose
convolution



Input:
 $3 \times H \times W$



Predictions:
 $H \times W$

Upsampling: Unpooling

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

Upsampling: Unpooling

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

“Bed of Nails”

1	2
3	4



1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

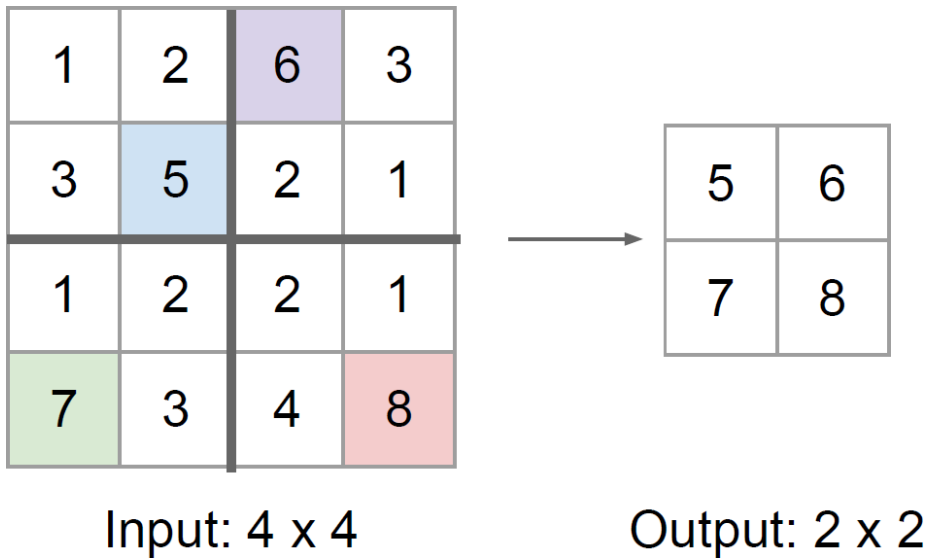
Input: 2 x 2

Output: 4 x 4

Upsampling: Max Unpooling

Max Pooling

Remember which element was max!



Upsampling: Max Unpooling

Max Pooling

Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4



5	6
7	8

Output: 2 x 2

Max Unpooling

Use positions from pooling layer

1	2
3	4

Input: 2 x 2



0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

Output: 4 x 4

Upsampling: Max Unpooling

Max Pooling

Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4



5	6
7	8

Output: 2 x 2

Max Unpooling

Use positions from pooling layer

1	2
3	4

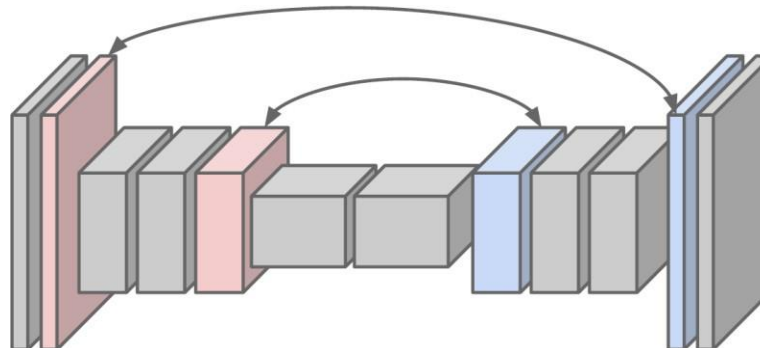
Input: 2 x 2



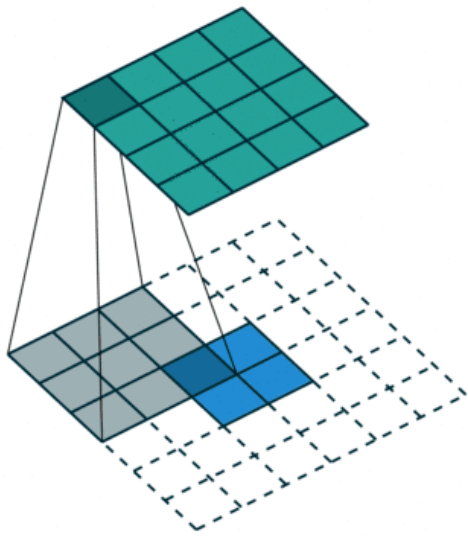
0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

Output: 4 x 4

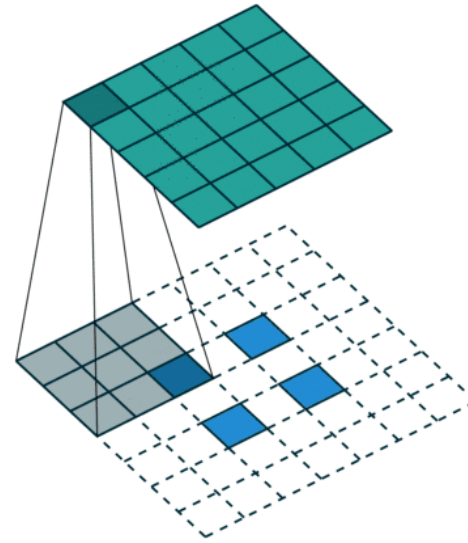
Corresponding pairs of
downsampling and
upsampling layers



Upsampling: Transpose Convolution

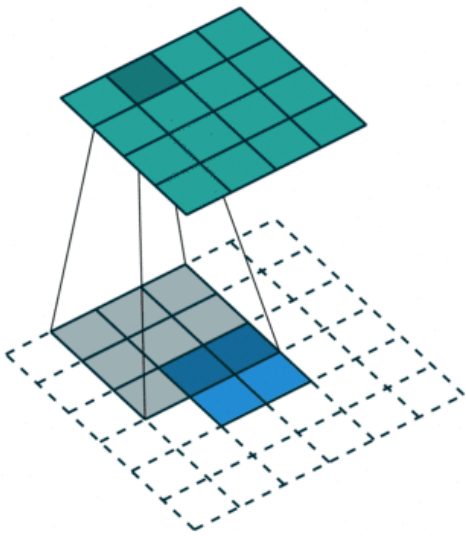


Upsampling 2×2 input to a 4×4 output

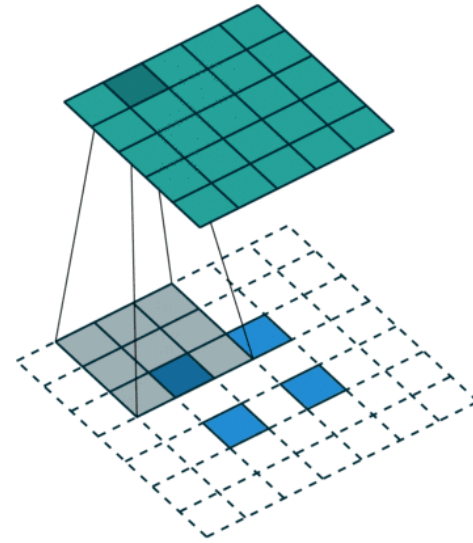


Upsampling 2×2 input to a 5×5 output

Upsampling : Transpose Convolution

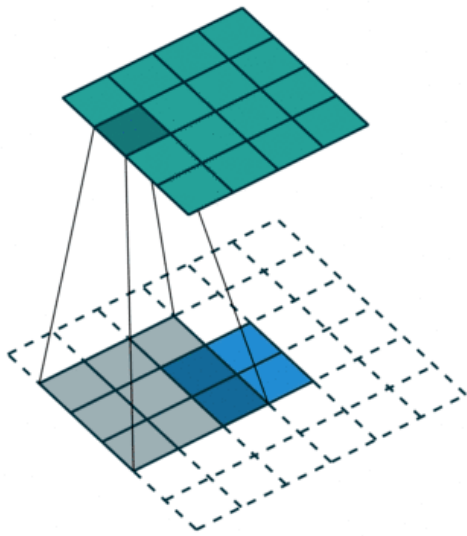


Upsampling 2×2 input to a 4×4 output

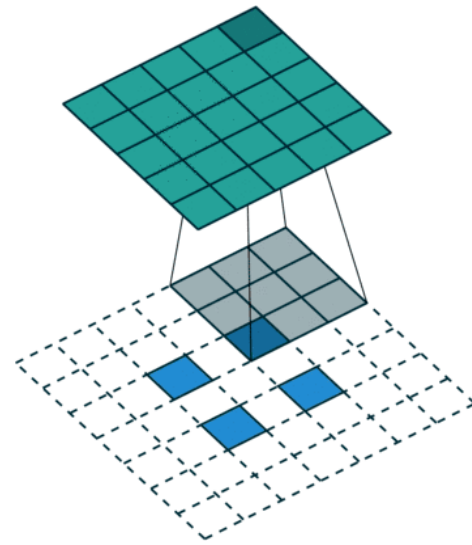


Upsampling 2×2 input to a 5×5 output

Upsampling : Transpose Convolution

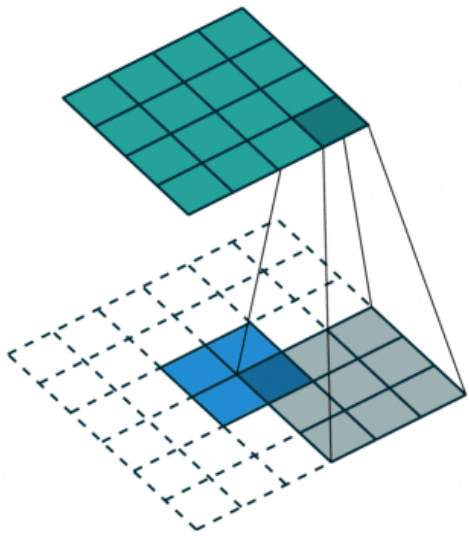


Upsampling 2×2 input to a 4×4 output

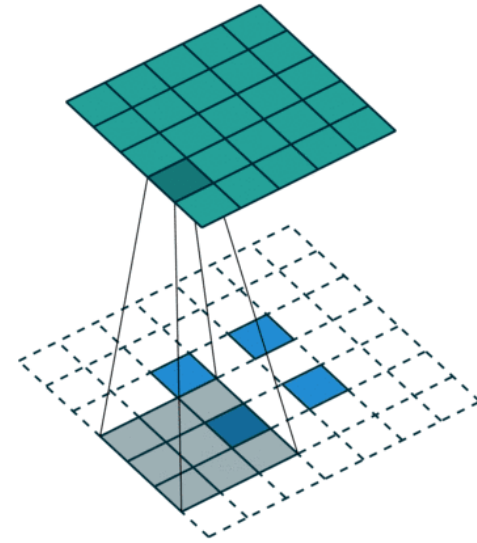


Upsampling 2×2 input to a 5×5 output

Upsampling : Transpose Convolution



Upsampling 2×2 input to a 4×4 output



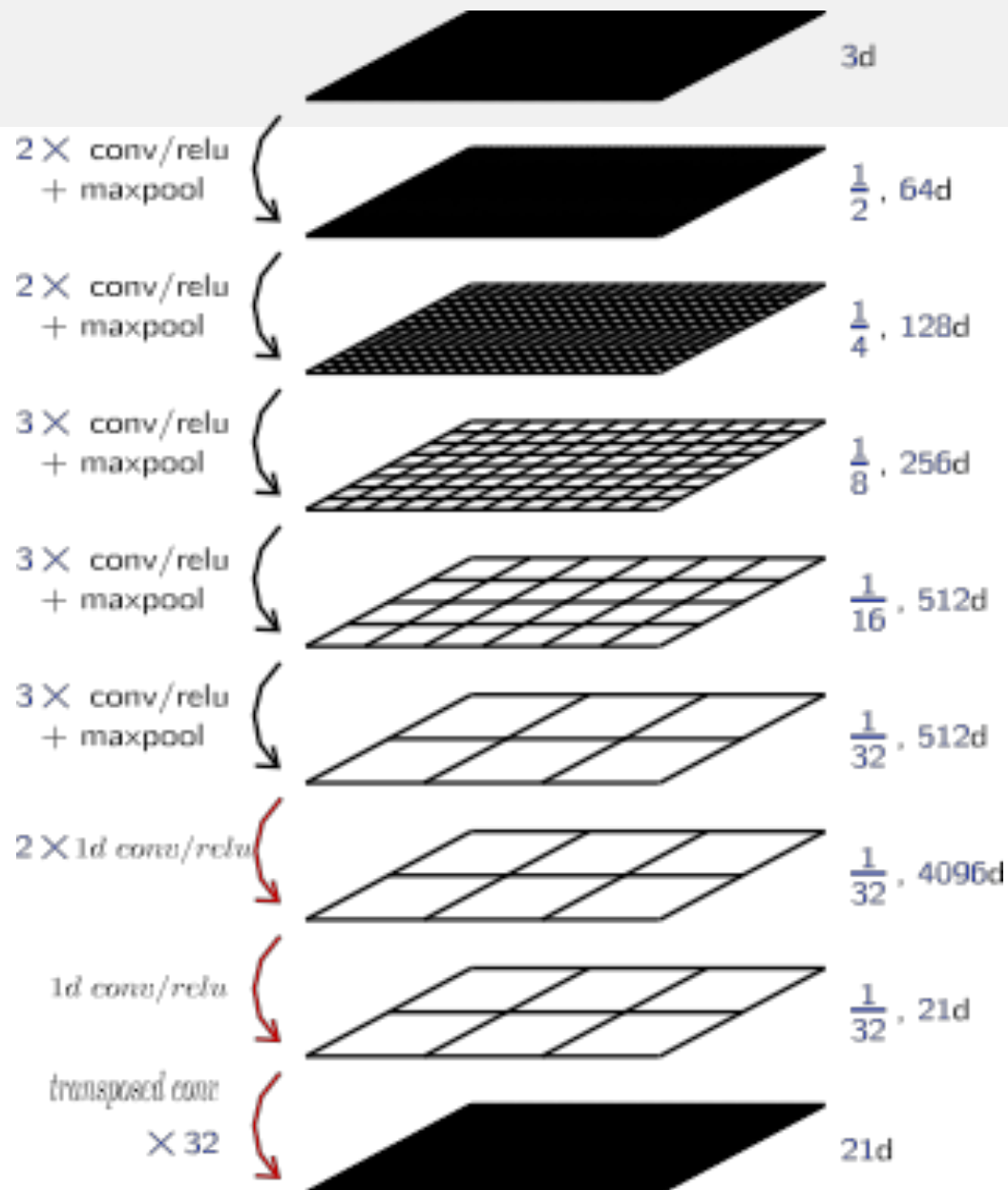
Upsampling 2×2 input to a 5×5 output

Fully Convolutional Networks for Semantic Segmentation (FCN)¹

FCN with VGG-16 Backbone

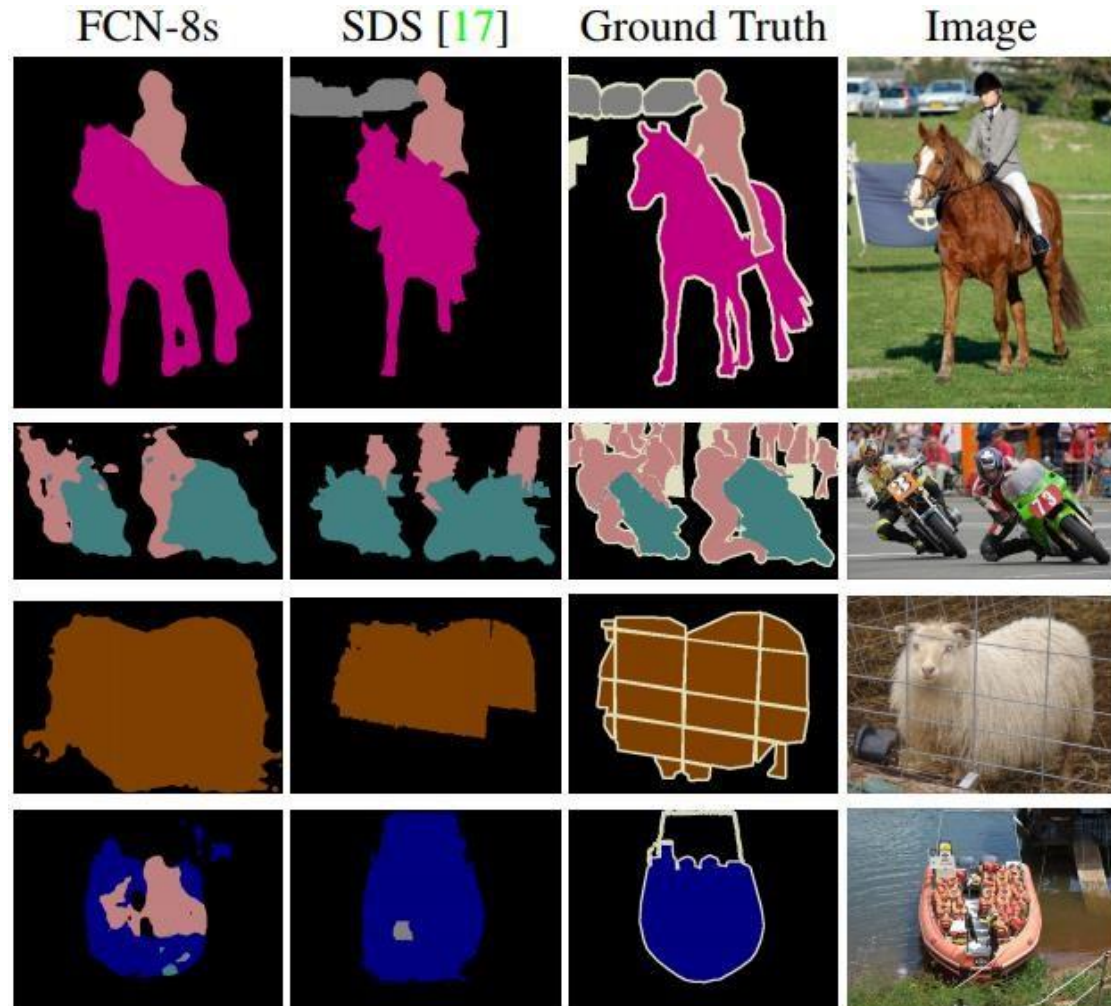
- Remove fully connected layers Replace three FC layers with 1×1 conv layers (1D) $\rightarrow$ Adapts various classification networks (VGG net, GoogLeNet) into FCN
- To obtain classification for each pixel, another 1×1 conv layer is appended with channel dimension $C + 1$ where C is number of classes $\rightarrow$ last layer has $C + 1$ filters which give class probabilities
- spatial size is downsampled because of maxpool operations $\rightarrow$ One final upsampling layer to get back to original size
- Instead of one final upsampling layer, can have skip connections from earlier layers for better boundaries

¹Shelhamer et al, Fully Convolutional Networks for Semantic Segmentation, TPAMI 2016

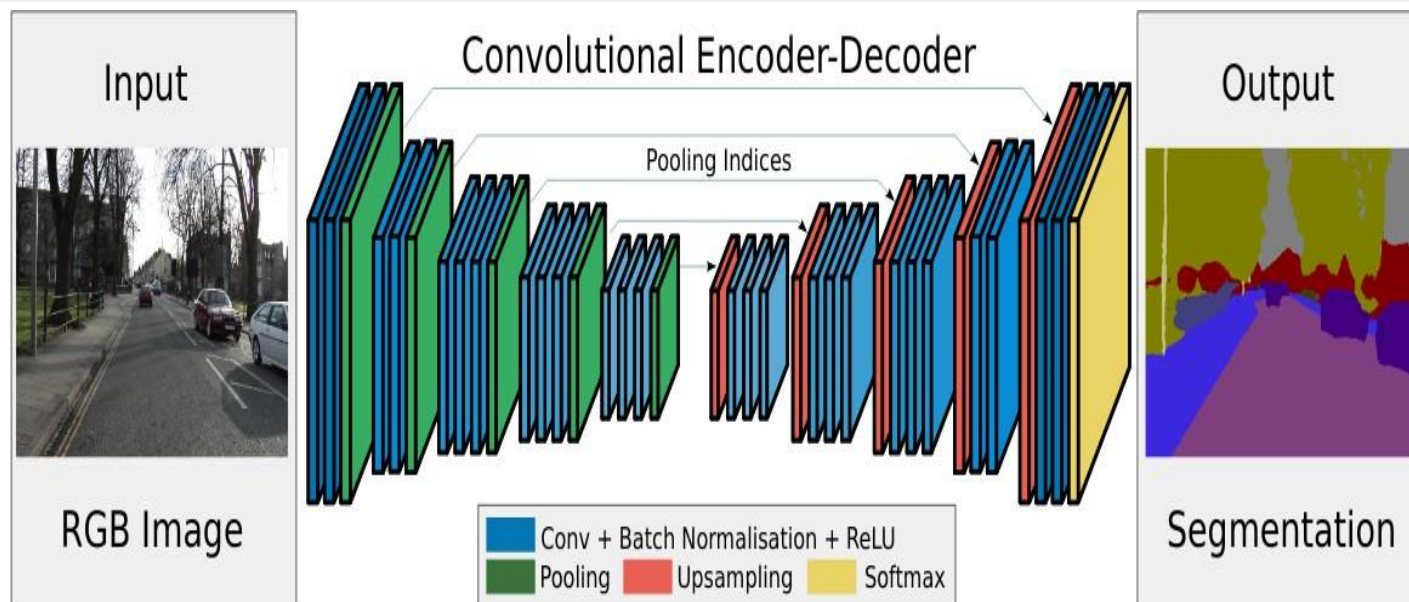


“Fully convolutional (FCN)” results

- Takes advantage of pre-training from classification
- Applied to objects and scenes (NYUd v2)
- But feature pooling reduces spatial sensitivity and resolution



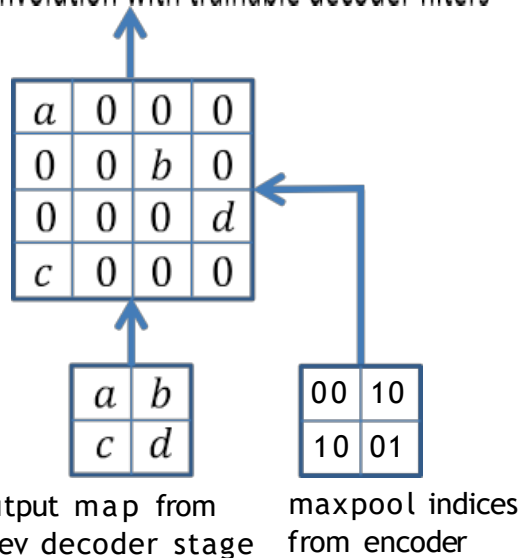
SegNet²



- Fully convolutional encoder-decoder architecture
- Encoder is VGG-16 without FC layers
- Decoder maps low-resolution encoder feature maps to input resolution
- Decoder uses pooling indices of corresponding max-pooling steps to perform upsampling!
- These sparse upsampled maps are followed by conv layers to produce dense feature maps

Upsampling in SegNet

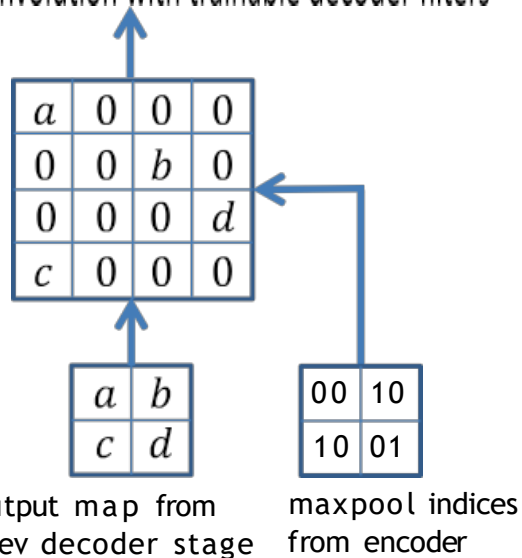
Convolution with trainable decoder filters



- **Max pool indices:** location of maximum feature value for each pooling window for each encoder feature map
 - Storing maxpool indices is memory-efficient (instead of storing full feature map)
 - In each stage of decoder, max pooling indices from corresponding encoder stage used to produce sparse upsampled feature maps
-
- What is the loss function in all these methods?

Upsampling in SegNet

Convolution with trainable decoder filters



- **Max pool indices:** location of maximum feature value for each pooling window for each encoder feature map
 - Storing maxpool indices is memory-efficient (instead of storing full feature map)
 - In each stage of decoder, max pooling indices from corresponding encoder stage used to produce sparse upsampled feature maps
-
- What is the loss function in all these methods? **Sum of pixel-wise cross-entropy losses**

Other Segmentation Architectures

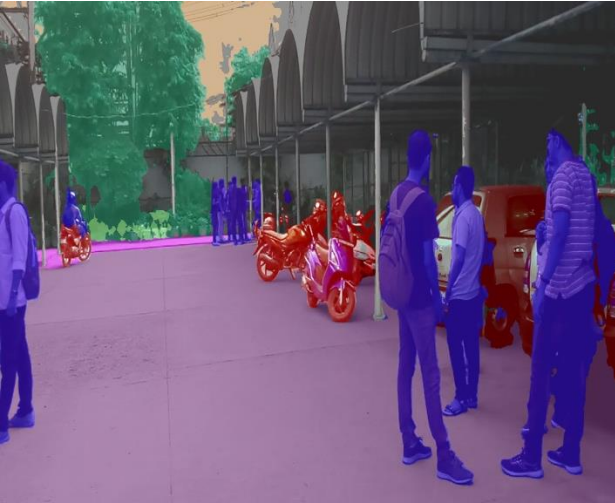
Semantic Segmentation

- UNet
- PSPNet: Pyramid Scene Parsing Network
- Deep Lab

Instance Segmentation

- Mask R-CNN

Panoptic Segmentation⁸



Semantic Segmentation



Instance segmentation



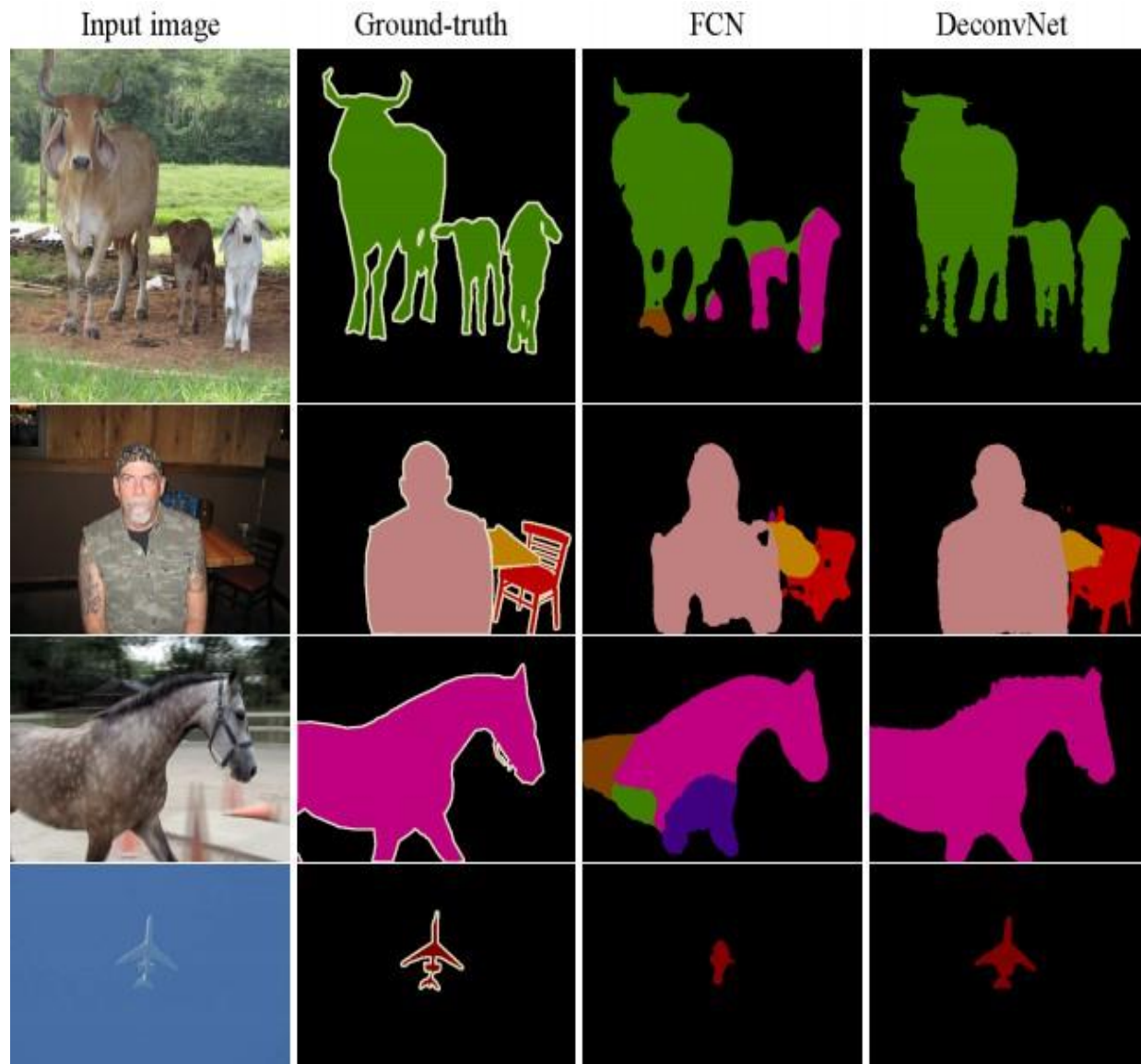
Panoptic segmentation

Credit: [Harshit Kumar, Github.io](https://github.com/HarshitKumar)¹

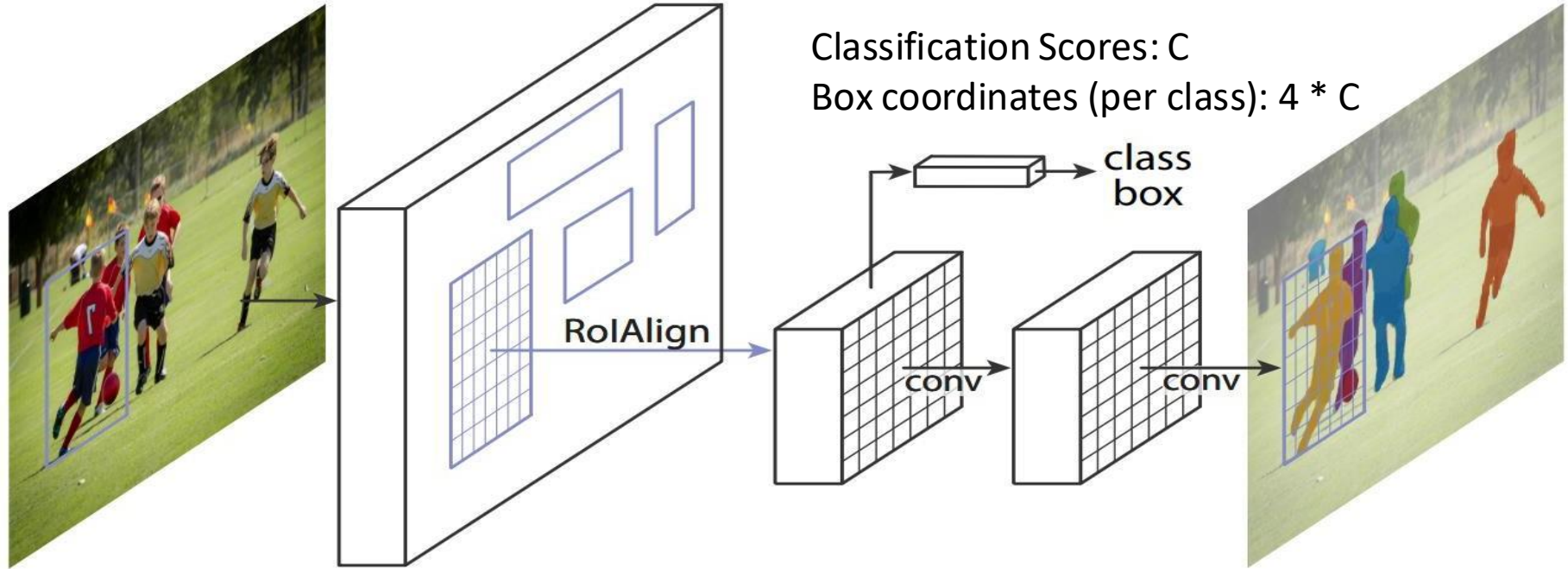
⁸Kirillov et al, Panoptic Segmentation, CVPR 2019

“DeconNet” results

- A deep deconvolution network
- Pixel-wise prediction



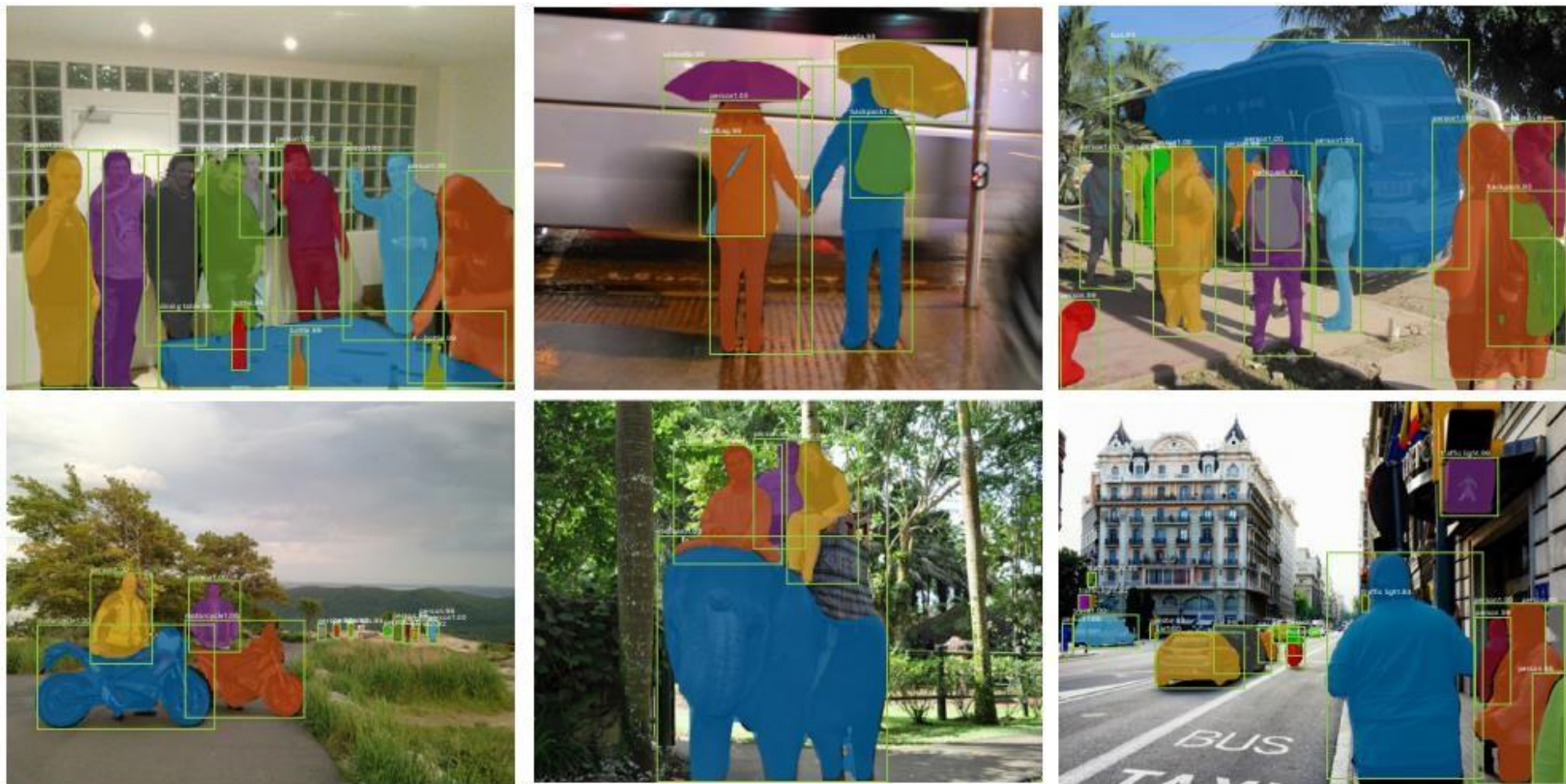
Instance Segmentation: Mask R-CNN (He et al. ICCV 2017)



Mask R-CNN - extends Faster R-CNN by adding a branch for predicting an object mask in parallel with the existing branch for bounding box recognition.

Mask R-CNN is simple to train and adds only a small overhead to Faster R-CNN, running at 5 fps.

Instance Segmentation: Mask R-CNN (He et al. ICCV 2017)



Mask R-CNN results on the COCO test set. These results are based on ResNet-101, achieving a mask AP of 35.7 and running at 5 fps. Masks are shown in color, and bounding box, category, and confidences are also shown.

He et al. Mask R-CNN, ICCV 2017. <https://arxiv.org/pdf/1703.06870.pdf>

Acknowledgements

- Thanks to the following researchers for making their teaching/research material online
 - Forsyth
 - Steve Seitz
 - Noah Snavely
 - J.B. Huang
 - Derek Hoiem
 - J. Hays
 - J. Johnson
 - R. Girshick
 - S. Lazebnik
 - K. Grauman
 - Antonio Torralba
 - Rob Fergus
 - Leibe
 - And many more